COP Upstream Platform — Complete Application Documentation

Version: 1.0.0 **Last Updated:** March 2026 **Environment:** Production — Azure (Central US) **Live URL:** https://cop-upstream-platform.azurewebsites.net

---

Table of Contents

1. Application Overview
2. Technology Stack
3. System Architecture
4. Application Modules
   - 4.1 Partner Onboarding
   - 4.2 Production Revenue Accounting (PRA)
   - 4.3 Joint Venture Accounting (JVA)
5. Database Design
6. Authentication & Authorization
7. API Reference
8. Frontend Architecture
9. Azure Deployment Architecture
10. CI/CD Pipeline
11. Configuration & Environment Variables
12. User Management
13. Data Flow Diagrams
14. Project Structure
15. Operational Runbook

---

1. Application Overview

The **COP Upstream Platform** is a web-based management system built for ConocoPhillips' upstream oil and gas operations. It consolidates three major business workflows into a single application:

Module	Purpose
Partner Onboarding	Structured 7-step workflow to onboard new joint venture partners
Production Revenue Accounting (PRA)	End-to-end revenue management — volumes, pricing, royalties, owner payments
Joint Venture Accounting (JVA)	AFE approvals, JIB billing, cash call management, and audit tracking

Who Uses This Application

Role	Access Level	Typical User
Admin	Full access including user management	IT Administrator, Controller
Analyst	Read + write data; cannot manage users	Revenue Analyst, JV Accountant
Viewer	Read-only access to all modules	Management, Auditors, Partners

Key Business Value

- Replaces manual spreadsheets and email-based workflows with a structured, auditable system
- Provides real-time visibility into partner onboarding status, revenue position, and JV financials
- Enforces COPAS (Council of Petroleum Accountants Societies) accounting standards
- Integrates AFE elections, JIB billings, and cash call tracking in one place

---

2. Technology Stack

Backend

Component	Technology	Version
Runtime	Node.js	v20 LTS
Web Framework	Express.js	^4.21.0
Database Client	pg (node-postgres)	^8.20.0
Authentication	jsonwebtoken	^9.0.3
Password Hashing	bcryptjs	^3.0.3
CORS	cors	^2.8.5
Config Management	dotenv	^17.3.1

Database

Component	Technology
Database Engine	PostgreSQL 16
Hosting	Azure Database for PostgreSQL Flexible Server
Connection	pg.Pool (connection pooling)

Frontend

Component	Technology
Architecture	Single Page Application (SPA)

Component	Technology
Language	Vanilla JavaScript (ES2020+)
Charts	Chart.js 4.4.1
Fonts	Barlow, Barlow Condensed, JetBrains Mono (Google Fonts)
Styling	Custom CSS with CSS Variables

Infrastructure

Component	Service
Application Hosting	Azure App Service (Linux, B1 tier)
Database	Azure Database for PostgreSQL Flexible Server
Source Control	GitHub (https://github.com/sriman2709/COPProject)
CI/CD	GitHub Actions

---

3. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        INTERNET / USERS                      │
└──────────────────────────────────────────────────────────────┘
                          │ HTTPS (443)
                          ▼
┌──────────────────────────────────────────────────────────────┐
│              Azure App Service (cop-upstream-platform)       │
│                      Central US — B1 Linux                   │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                    Express.js Server                 │   │
│   │                       (Port 3000)                    │   │
│   │                                                      │   │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
```

```
│   │     │ Static SPA  │    │ JWT Auth     │    │ CORS         │   │   │
│   │     │ /public     │    │ Middleware   │    │ Middleware   │   │   │
│   │     └─────────────┘    └──────────────┘    └──────────────┘   │   │
│   │                                                               │   │
│   │     ┌─────────────────────────────────────────────────────┐   │   │
│   │     │                    API Routes                       │   │   │
│   │     │ /api/auth  /api/partners  /api/onboarding           │   │   │
│   │     │ /api/documents  /api/dashboard  /api/pipeline       │   │   │
│   │     │ /api/pra   /api/jva                                 │   │   │
│   │     └─────────────────────────────────────────────────────┘   │   │
│   │                           │                                   │   │
│   │     ┌─────────────────────▼───────────────────────────────┐   │   │
│   │     │                 Data Models (30+)                   │   │   │
│   │     │ Partner, Onboarding, AFE, JIB, CashCall,            │   │   │
│   │     │ RoyaltyRun, ProductionVolume, PriceIndex...         │   │   │
│   │     └─────────────────────────────────────────────────────┘   │   │
│   └───────────────────────────────────────────────────────────────┘   │
└───────────────────────────────────────────────────────────────────────┘
                                │
                                │ PostgreSQL SSL (5432)
                                ▼
┌───────────────────────────────────────────────────────────────────────┐
│         Azure Database for PostgreSQL Flexible Server                 │
│           cop-postgres-srv.postgres.database.azure.com                │
│                  Standard_B1ms — PostgreSQL 16                        │
│                                                                       │
│    ┌──────────────────────────────────────────────────────────┐       │
│    │                   copproject database                    │       │
│    │   40+ tables: partners, onboarding_sessions, afes,       │       │
│    │   jibs, cash_calls, royalty_runs, pra_periods...         │       │
│    └──────────────────────────────────────────────────────────┘       │
└───────────────────────────────────────────────────────────────────────┘
```

Request Flow

```
Browser Request
    │
    ▼
Azure App Service (HTTPS termination)
    │
    ▼
Express.js Router
    │
    ├── Static files (.html/.css/.js) → served directly from /public
    │
    └── /api/* routes
            │
            ▼
        authenticate() middleware
        (validates JWT Bearer token)
            │
            ▼
        requireAdmin() / requireAnalyst()
        (role enforcement, if applicable)
            │
```

```
            ▼
        Route Handler (async/await)
            │
            ▼
        Data Model Method
        (pg.Pool query to PostgreSQL)
            │
            ▼
        JSON Response → Browser
```

---

4. Application Modules

4.1 Partner Onboarding

The onboarding module manages the complete lifecycle of bringing a new joint venture partner into ConocoPhillips' systems through a structured 7-step workflow.

The 7-Step Onboarding Process

```
Step 1          Step 2          Step 3          Step 4
Company    →    JOA &      →    Financial  →    COPAS
Registration    Working Int.    Setup           Compliance
                                                    │
                                                    ▼
Step 7          Step 6          Step 5
Review &   ←    System     ←    Document
Submit          Access          Collection
```

Step 1 — Company Registration

- Captures legal entity name, DBA name, entity type (LLC/Corporation/LP/LLP/Trust/Individual)
- EIN/TIN and DUNS number
- State of formation and formation date
- Business address
- Primary and secondary contacts (operations, finance, legal, billing, elections roles)

Step 2 — JOA & Working Interest

- Creates the Joint Operating Agreement (JOA) record
- Links properties/assets to the JOA (basin, product type)
- Defines Working Interest (WI%) and Net Revenue Interest (NRI%) percentages

- Calculates summary totals for reporting

Step 3 — Financial Setup

- Bank account information (routing number, masked account number, account type)
- Payment method: ACH/EFT preferred
- Tax classification for 1099-MISC Box 2 reporting
- W9 receipt status
- ACH prenote status and tracking
- Delivery preferences: JIB and cash call delivery methods (email/EDI/portal)
- Revenue distribution method
- Contact emails for cash calls and AFE elections

Step 4 — COPAS Compliance

- COPAS version selection (2005 MFI-51 standard)
- Overhead method (Fixed Rate per Well)
- Escalation index selection (COPAS Index)
- Material pricing method
- Audit period and dispute resolution
- Partner acknowledgment with timestamp

Step 5 — Document Collection

- Tracks required documents: W9, Tax ID documentation, Bank verification
- Document statuses: pending → received → verified → complete
- Overdue detection based on required dates
- Verification workflow with analyst review

Step 6 — System Access Provisioning

- Creates partner portal user account
- Configures access levels: read-only or elections-enabled
- EDI gateway setup: JIB 810 (billing), Cash Call 820 (funding)
- Mineral Connect and AFE Election Portal enablement
- Notification configuration: JIB, Cash Call, AFE, Revenue notifications

Step 7 — Review & Submit

- Final review of all 6 preceding steps
- Sign-off and completion
- Triggers status change from `active` to `completed`

Progress Calculation

```
Progress % = (completed_steps × 100 + active_steps × 45) / 7
```

Pipeline View

Provides a portfolio view of all active onboarding sessions with:

- Status categorization (active / in review / final review / overdue)
- Target date tracking
- Monthly completion trends

4.2 Production Revenue Accounting (PRA)

The PRA module manages the full cycle of production revenue from wellhead volumes through to royalty owner payments.

PRA Business Process Flow

```
Production          Allocation        Pricing
Volumes       →     & Balancing  →    & Contracts
(SCADA input)       (well/facility)    (price locking)
                                            │
                                            ▼
Owner               Royalty           Remittance
Payments      ←     Run           ←   Matching
(ACH/Check)         (calculate)        (validate)
```

Volume & Measurement

- **Production Volumes**: Monthly production data (oil BBL, gas MCF, NGL BBL) from SCADA systems, validated against expected ranges with exception flagging
- **Well Allocation**: Facility-level volume balancing; tracks purchaser-reported vs. allocated volumes with variance percentage alerts

- **Gas Balancing**: Entitlement vs. taken position tracking; identifies overlift/underlift situations; calculates balance sheet value and makeup expiry dates

Pricing & Contracts

- **Price Index Management**: Tracks commodity indices (WTI Cushing, Henry Hub, etc.) with history; analysts lock prices before royalty calculations
- **Marketing Contracts**: Master contract records with purchaser, pricing formula, delivery point, and volume commitments
- **Remittance Matching**: Compares company volumes/prices to purchaser remittance advice; flags volume variance (>2%) and price variances for resolution; un-matched amounts flow to suspense
- **Suspense Accounts**: Holds revenue pending owner identification or dispute resolution; tracks age and escheat risk

Royalty & Owners

- **Division Orders**: Decimal interest records for each royalty owner by property; validates that interests sum to 100%; tracks pending/suspended status
- **Royalty Calculation**: Period-based calculations applying price to allocated volumes against division order interests; respects deductibility elections
- **Owner Statements**: Payment detail by owner across all properties

Revenue Distribution

- **Royalty Runs**: Batch payment processing; tracks check/EFT counts, below-minimum payees, decimal errors
- **Owner Payments**: Individual payment records with ACH/check method; held payments for suspense items

---

4.3 Joint Venture Accounting (JVA)

The JVA module handles all financial interactions between ConocoPhillips as operator and non-operating working interest partners.

JVA Business Process Flow

```
AFE                Partner           JIB
```

```
Authorization →   Elections     →   Preparation
(project budget)  (participate/     (actual billing)
                  non-consent)          |
                                        ▼
Audit             Cash Call        Receipt
Management   ←    Funding      ←   Application
(findings)        (calls/overdue)   (payments)
```

AFE Management (Authorization For Expenditure)

- **AFE Creation**: Capital project approval requests with COPAS cost code breakdown (100-Labor, 200-Materials, 300-Transport, 400-Services, 600-Overhead, 700-Taxes)
- **AFE Types**: Drilling, Workover, Facility, Plug & Abandon
- **Budget Tracking**: Authorized amount vs. actual spend with percentage tracking
- **NOP Elections**: Non-operating partner responses (participate / non-consent); tracks response deadlines; defaulted elections trigger non-consent penalty calculations
- **Cost Tracker**: Real-time budget vs. actual comparison by COPAS category

JIB Preparation (Joint Interest Billing)

- Monthly billing of non-operators for their share of operating costs
- Direct cost allocation based on working interest percentage
- Overhead calculation per COPAS schedule (Fixed Rate per Well)
- Line item breakdown by COPAS cost code
- EDI 810 delivery to partner systems
- Payment tracking: issued → partial → paid → overdue

Cash Call Management

- Advance funding requests for upcoming capital or operating expenditures
- Due date enforcement with overdue detection
- Interest accrual on overdue amounts
- Receipt matching and application against outstanding calls
- Forecasting for upcoming cash requirements

Audit Management

- Audit finding documentation with COPAS category classification
- Dollar amount under audit

- Response deadline tracking
- Status lifecycle: open → disputed → resolved / credit_due

---

5. Database Design

Database Overview

- **Engine**: PostgreSQL 16
- **Database Name**: copproject
- **Total Tables**: 40+
- **Encoding**: UTF8 with en_US.utf8 collation

Schema Groups

Authentication (1 table)

Table	Purpose
`auth_users`	Platform user accounts with bcrypt-hashed passwords and role assignments

Partner Onboarding (8 tables)

Table	Purpose
`partners`	Core partner entity data (legal name, address, EIN/TIN)
`contacts`	Partner contacts by role (operations/finance/legal/billing/elections)
`joa`	Joint Operating Agreement records
`properties`	Properties/assets linked to JOAs
`working_interests`	WI% and NRI% assignments per partner per property
`financial_setup`	Banking, ACH, tax, and delivery preferences
`copas_config`	COPAS overhead and accounting procedure configuration

Table	Purpose
`overhead_rates`	COPAS overhead rate tiers by category
`onboarding_sessions`	Workflow state: 7-step progress, statuses, target dates
`documents`	Required document tracking with receipt/verification workflow
`user_accounts`	Partner portal access accounts
`activity_log`	Audit trail for onboarding activities

PRA — Production Revenue Accounting (11 tables)

Table	Purpose
`pra_periods`	Accounting period lifecycle (open/locked/closed) with close checklist
`production_volumes`	Monthly volumes (oil/gas/NGL) from SCADA by property
`well_allocations`	Facility allocation records with purchaser variance tracking
`gas_imbalances`	Overlift/underlift balance sheet by property
`price_indices`	Commodity price indices with lock workflow
`marketing_contracts`	Purchaser sales contracts with pricing formulas
`remittances`	Purchaser payment advice matching records
`suspense_accounts`	Revenue held pending owner resolution
`division_orders`	Royalty owner decimal interest records
`royalty_runs`	Monthly royalty calculation batch records
`owner_payments`	Individual owner payment details per royalty run

JVA — Joint Venture Accounting (10 tables)

Table	Purpose
`afes`	AFE capital project authorization records
`afe_line_items`	COPAS-coded cost budget line items per AFE

Table	Purpose
`afe_elections`	Partner election responses per AFE
`jibs`	Monthly non-operator billing records
`jib_line_items`	COPAS-coded cost line items per JIB
`cash_calls`	Partner advance funding requests
`receipts`	Cash receipts from partners
`audit_findings`	Audit exceptions by property and COPAS category

Key Relationships

```
partners (1) ──────── (N) contacts
partners (1) ──────── (1) financial_setup
partners (1) ──────── (1) copas_config
partners (1) ──────── (N) joa
partners (1) ──────── (1) onboarding_sessions

joa (1) ──────────── (N) properties
properties (1) ─── (N) working_interests
properties (1) ─── (N) production_volumes
properties (1) ─── (N) afes
properties (1) ─── (N) jibs
properties (1) ─── (N) cash_calls

afes (1) ──────────── (N) afe_line_items
afes (1) ──────────── (N) afe_elections
jibs (1) ──────────── (N) jib_line_items
royalty_runs (1) ─── (N) owner_payments
```

Database Initialization

On first server startup, `initDb()` in `database/db.js`:

1. Checks if the `partners` table exists
2. If not — runs `schema.sql` to create all tables
3. Runs `seed.sql` to load demo/reference data
4. Runs `auth_schema.sql` (CREATE TABLE IF NOT EXISTS — safe to re-run)
5. Creates default admin user if `auth_users` is empty

---

6. Authentication & Authorization

Authentication Method

JSON Web Tokens (JWT)

- Algorithm: HS256
- Expiration: 8 hours (configurable via `JWT_EXPIRES_IN`)
- Transport: HTTP Authorization header — `Bearer <token>`
- Storage: Browser localStorage

Login Flow

```
1. User submits email + password to POST /api/auth/login

2. Server looks up user in auth_users table

3. bcryptjs.compare() validates password against stored hash

4. On success: JWT signed with {id, name, email, role}
   Token returned to browser

5. Browser stores token in localStorage as 'cop_token'

6. All subsequent API requests include:
   Authorization: Bearer <token>

7. Server middleware validates token on every API call

8. On 401 (expired/invalid): browser clears token, shows login screen
```

Role-Based Access Control

```
Role Hierarchy:
┌──────────────────────────────────────────┐
│  ADMIN                                   │
│  ┌──────────────────────────────────┐    │
│  │  ANALYST                         │    │
│  │  ┌──────────────────────────┐    │    │
│  │  │  VIEWER                  │    │    │
│  │  │  (authenticated read-only)│    │    │
│  │  └──────────────────────────┘    │    │
│  └──────────────────────────────────┘    │
└──────────────────────────────────────────┘
```

Action	Viewer	Analyst	Admin
Read all data (GET)	✅	✅	✅
Create/update data (POST/PUT)	❌	✅	✅
Delete data (DELETE)	❌	❌	✅
Register new users	❌	❌	✅
View all users	❌	❌	✅
Change user roles	❌	❌	✅
Change own password	✅	✅	✅

Middleware Implementation

```
// Applied to ALL routes in every route file:
router.use(authenticate);        // requires valid JWT

// Applied to write operations:
router.post('/', requireAnalyst, handler);   // admin or analyst

// Applied to delete/admin operations:
router.delete('/:id', requireAdmin, handler); // admin only
```

Password Security

- Passwords hashed using **bcryptjs with 12 salt rounds**
- Plaintext passwords are never stored or logged
- Password changes require the current password to be provided

---

7. API Reference

Base URL

- **Production**: `https://cop-upstream-platform.azurewebsites.net`
- **Local**: `http://localhost:3000`

Authentication Header

All protected endpoints require:

```
Authorization: Bearer <JWT token>
```

HTTP Status Codes

Code	Meaning
200	Success
201	Created
401	Unauthorized (missing/invalid/expired token)
403	Forbidden (insufficient role)
404	Resource not found
500	Server error (includes error message)

Auth Endpoints — `/api/auth`

Method	Endpoint	Auth	Role	Description
POST	`/login`	No	—	Login; returns `{token}`
GET	`/me`	Yes	Any	Returns current user info

Method	Endpoint	Auth	Role	Description
POST	`/register`	Yes	Admin	Create new user `{name, email, password, role}`
GET	`/users`	Yes	Admin	List all platform users
PUT	`/users/:id`	Yes	Admin	Update user fields
PUT	`/password`	Yes	Any	Change own password
DELETE	`/users/:id`	Yes	Admin	Delete user

Partner Endpoints — `/api/partners`

Method	Endpoint	Auth	Role	Description
GET	`/`	Yes	Any	List all partners
POST	`/`	Yes	Analyst+	Create partner
GET	`/:id`	Yes	Any	Get partner
PUT	`/:id`	Yes	Analyst+	Update partner
GET	`/:id/full`	Yes	Any	Partner with all related data
GET	`/:id/contacts`	Yes	Any	List contacts
POST	`/:id/contacts`	Yes	Analyst+	Add contact
PUT	`/:pid/contacts/:cid`	Yes	Analyst+	Update contact
DELETE	`/:pid/contacts/:cid`	Yes	Admin	Delete contact
GET	`/:id/joa`	Yes	Any	JOA with working interests
POST	`/:id/joa`	Yes	Analyst+	Create JOA
PUT	`/:pid/joa/:jid`	Yes	Analyst+	Update JOA
GET	`/:id/working-interests`	Yes	Any	Working interests
GET	`/:id/wi-summary`	Yes	Any	WI summary stats

Method	Endpoint	Auth	Role	Description
GET	`/:id/financial`	Yes	Any	Financial setup
PUT	`/:id/financial`	Yes	Analyst+	Update financial
GET	`/:id/copas`	Yes	Any	COPAS config
PUT	`/:id/copas`	Yes	Analyst+	Update COPAS
POST	`/:id/copas/acknowledge`	Yes	Analyst+	Acknowledge COPAS
GET	`/:id/users`	Yes	Any	Portal users
POST	`/:id/users`	Yes	Analyst+	Create portal user
PUT	`/:pid/users/:uid`	Yes	Analyst+	Update portal user
DELETE	`/:pid/users/:uid`	Yes	Admin	Delete portal user

Onboarding Endpoints — `/api/onboarding`

Method	Endpoint	Auth	Role	Description
GET	`/`	Yes	Any	List all sessions
POST	`/`	Yes	Analyst+	Create session
GET	`/:id`	Yes	Any	Get session
PUT	`/:id`	Yes	Analyst+	Update session
PUT	`/:id/step/:step`	Yes	Analyst+	Update step status
POST	`/:id/advance/:step`	Yes	Analyst+	Mark step done, advance next
GET	`/:id/progress`	Yes	Any	Step progress
GET	`/:id/activity`	Yes	Any	Activity log
POST	`/:id/activity`	Yes	Any	Add activity entry

Dashboard Endpoint — `/api/dashboard`

Method	Endpoint	Auth	Role	Description
GET	`/:onboardingId`	Yes	Any	Full dashboard data: KPIs, partner info, step statuses, doc stats, activity timeline, JOAs, WI summary

---

Pipeline Endpoints — `/api/pipeline`

Method	Endpoint	Auth	Role	Description
GET	`/stats`	Yes	Any	Pipeline stats: counts by status, monthly completions
GET	`/`	Yes	Any	All sessions with overdue flag

Document Endpoints — `/api/documents`

Method	Endpoint	Auth	Role	Description
GET	`/onboarding/:id`	Yes	Any	Documents for a session
GET	`/onboarding/:id/stats`	Yes	Any	Document completion stats
GET	`/:id`	Yes	Any	Get document
PUT	`/:id`	Yes	Analyst+	Update document
POST	`/:id/receive`	Yes	Analyst+	Mark received
POST	`/:id/verify`	Yes	Analyst+	Mark verified
POST	`/`	Yes	Analyst+	Create document

PRA Endpoints — `/api/pra`

Method	Endpoint	Auth	Role	Description
GET	`/dashboard`	Yes	Any	Full PRA dashboard
GET	`/periods`	Yes	Any	All periods
GET	`/periods/current`	Yes	Any	Current open period
PUT	`/periods/:id`	Yes	Analyst+	Update period
PUT	`/periods/:id/checklist`	Yes	Analyst+	Update close checklist
GET	`/volumes/:period`	Yes	Any	Volumes by period
GET	`/volumes/exceptions/:period`	Yes	Any	Volume exceptions
GET	`/volumes/property/:id`	Yes	Any	Volumes by property
POST	`/volumes`	Yes	Analyst+	Create volume record
PUT	`/volumes/:id`	Yes	Analyst+	Update volume
GET	`/allocations/:period`	Yes	Any	Allocations by period
GET	`/allocations/alerts`	Yes	Any	Variance alerts
GET	`/allocations/facility/:name`	Yes	Any	Allocations by facility
POST	`/allocations`	Yes	Analyst+	Create allocation
PUT	`/allocations/:id`	Yes	Analyst+	Update allocation
GET	`/gas-balances`	Yes	Any	All gas imbalances
GET	`/gas-balances/alerts`	Yes	Any	Imbalance alerts

Method	Endpoint	Auth	Role	Description
GET	`/gas-balances/property/:id`	Yes	Any	Balances by property
PUT	`/gas-balances/:id`	Yes	Analyst+	Update balance
GET	`/prices/:period`	Yes	Any	Price indices by period
GET	`/prices/history/:indexName`	Yes	Any	Price history (12 months)
POST	`/prices`	Yes	Analyst+	Create price index
POST	`/prices/:id/lock`	Yes	Analyst+	Lock price
GET	`/contracts`	Yes	Any	All contracts
GET	`/contracts/expiring`	Yes	Any	Expiring contracts
GET	`/contracts/:id`	Yes	Any	Contract details
POST	`/contracts`	Yes	Analyst+	Create contract
PUT	`/contracts/:id`	Yes	Analyst+	Update contract
GET	`/remittances/:period`	Yes	Any	Remittances by period
GET	`/remittances/stats/:period`	Yes	Any	Remittance stats
GET	`/remittances/exceptions/:period`	Yes	Any	Exceptions
POST	`/remittances`	Yes	Analyst+	Create remittance
PUT	`/remittances/:id`	Yes	Analyst+	Update remittance
GET	`/suspense`	Yes	Any	Suspense accounts

Method	Endpoint	Auth	Role	Description
GET	`/suspense/stats`	Yes	Any	Suspense stats
PUT	`/suspense/:id`	Yes	Analyst+	Update account
GET	`/division-orders`	Yes	Any	All division orders
GET	`/division-orders/errors`	Yes	Any	Orders with errors
GET	`/division-orders/stats`	Yes	Any	Stats
GET	`/division-orders/property/:id`	Yes	Any	Orders by property
PUT	`/division-orders/:id`	Yes	Analyst+	Update order
GET	`/royalty-runs`	Yes	Any	All royalty runs
GET	`/royalty-runs/latest`	Yes	Any	Latest run
GET	`/royalty-runs/:id`	Yes	Any	Run details
GET	`/royalty-runs/:id/summary`	Yes	Any	Run with payments
POST	`/royalty-runs`	Yes	Analyst+	Create run
GET	`/payments/:runId`	Yes	Any	Payments for run
GET	`/payments/:runId/stats`	Yes	Any	Payment stats

JVA Endpoints — `/api/jva`

Method	Endpoint	Auth	Role	Description
GET	`/dashboard`	Yes	Any	Full JVA dashboard
GET	`/afes`	Yes	Any	All AFEs

Method	Endpoint	Auth	Role	Description
GET	`/afes/:id`	Yes	Any	AFE details
POST	`/afes`	Yes	Analyst+	Create AFE
PUT	`/afes/:id`	Yes	Analyst+	Update AFE
GET	`/afes/:id/budget`	Yes	Any	AFE with line items
GET	`/afes/:id/lines`	Yes	Any	Line items
POST	`/afes/:id/lines`	Yes	Analyst+	Add line item
PUT	`/afe-lines/:id`	Yes	Analyst+	Update line item
GET	`/afes/:id/elections`	Yes	Any	Partner elections
GET	`/afes/:id/cost-tracker`	Yes	Any	Cost tracker
GET	`/elections`	Yes	Any	All elections
GET	`/elections/open`	Yes	Any	Open elections
PUT	`/elections/:id`	Yes	Analyst+	Record response
GET	`/jibs`	Yes	Any	All JIBs
GET	`/jibs/stats`	Yes	Any	JIB stats
GET	`/jibs/period/:period`	Yes	Any	JIBs by period
GET	`/jibs/:id`	Yes	Any	JIB with line items
POST	`/jibs`	Yes	Analyst+	Create JIB
PUT	`/jibs/:id`	Yes	Analyst+	Update JIB
GET	`/jibs/:id/lines`	Yes	Any	JIB line items
POST	`/jibs/:id/lines`	Yes	Analyst+	Add line item
GET	`/cash-calls`	Yes	Any	All cash calls
GET	`/cash-calls/overdue`	Yes	Any	Overdue calls
GET	`/cash-calls/stats`	Yes	Any	Cash call stats
GET	`/cash-calls/forecast`	Yes	Any	Forecast
POST	`/cash-calls`	Yes	Analyst+	Create cash call

Method	Endpoint	Auth	Role	Description
PUT	`/cash-calls/:id`	Yes	Analyst+	Update call
GET	`/receipts`	Yes	Any	All receipts
GET	`/receipts/pending`	Yes	Any	Pending receipts
POST	`/receipts`	Yes	Analyst+	Create receipt
POST	`/receipts/:id/apply`	Yes	Analyst+	Apply to cash call
GET	`/audit`	Yes	Any	All findings
GET	`/audit/stats`	Yes	Any	Audit stats
GET	`/audit/:id`	Yes	Any	Finding details
POST	`/audit`	Yes	Analyst+	Create finding
PUT	`/audit/:id`	Yes	Analyst+	Update finding

Total API Endpoints: 130+

---

8. Frontend Architecture

Single Page Application (SPA) Design

The entire frontend is contained in a single file: `public/index.html` (~2,600 lines).

Key Components

Authentication Layer

```
AUTH.getToken()      // reads JWT from localStorage
AUTH.getUser()       // decodes JWT payload (no server call)
AUTH.logout()        // clears token, shows login screen
doLogin()            // calls POST /api/auth/login, stores token
```

API Layer

```
API.get(url)          // GET with Authorization header
API.post(url, data)   // POST with Authorization + JSON body
API.put(url, data)    // PUT with Authorization + JSON body
// All methods automatically:
// - Attach Bearer token from localStorage
```

```
// - Detect 401 responses and force logout
```

State Management

```
const STATE = {
  partnerId: 1,         // currently viewed partner
  onboardingId: 1,      // currently viewed onboarding session
  dashboard: null,      // cached dashboard data
  activePage: 'ob-dashboard'
}
```

Navigation System

- 35+ named pages across 3 module groups
- `nav(pageId)` function renders the appropriate page
- Breadcrumb updates on navigation
- Active state tracked in sidebar

Page Rendering

- Each page is an async function (`pgDashboard`, `pgPipeline`, etc.)
- Returns HTML string; injected into `#contentArea`
- Charts initialized after DOM render via `requestAnimationFrame`

CSS Design System

CSS custom properties (variables) define the entire design language:

- Colors: COP Red (`#D42B2B`), Steel, Navy sidebar
- Spacing: consistent radius (`--r1` through `--r4`)
- Shadows: `--shadow-sm`, `--shadow`, `--shadow-lg`
- Typography: Barlow (body), Barlow Condensed (headings/KPIs), JetBrains Mono (numbers/codes)

Charts

Chart.js 4.4.1 renders 8 chart types across the application:

- Doughnut: Pipeline status breakdown, Revenue by product
- Bar: Monthly completions, AFE budget vs. actual, COPAS category costs
- Line: Revenue trend, Price history (dual-axis)

---

9. Azure Deployment Architecture

Azure Resources

Resource	Name	SKU/Tier	Region
Resource Group	`cop-platform-rg`	—	Central US
App Service Plan	`cop-app-plan`	B1 (Basic) Linux	Central US
Web App	`cop-upstream-platform`	Node 20 LTS	Central US
PostgreSQL Server	`cop-postgres-srv`	Standard_B1ms Burstable	Central US
Database	`copproject`	UTF8, en_US.utf8	—

App Service Configuration

Environment Variables (Application Settings):

Setting	Value	Purpose
`DATABASE_URL`	`postgresql://copadmin:***@cop-postgres-srv.postgres.database.azure.com/copproject?sslmode=require`	PostgreSQL connection
`JWT_SECRET`	(random 64-char hex)	JWT signing key
`JWT_EXPIRES_IN`	`8h`	Token lifetime
`NODE_ENV`	`production`	Runtime mode
`WEBSITE_NODE_DEFAULT_VERSION`	`~20`	Node version

Setting	Value	Purpose
`SCM_DO_BUILD_DURING_DEPLOYMENT`	`false`	Use pre-built node_modules

Startup Command: `node server.js`

PostgreSQL Firewall Rules

Rule	IP Range	Purpose
`AllowAllAzureServicesAndResourcesWithinAzureIps`	0.0.0.0	Allows App Service to connect

All connections require **SSL** (`sslmode=require` in connection string).

Network Topology

```
Internet Users
      │ HTTPS 443
      ▼
cop-upstream-platform.azurewebsites.net
(Azure App Service — Public endpoint)
      │ PostgreSQL 5432 (SSL)
      │ Azure internal network
      ▼
cop-postgres-srv.postgres.database.azure.com
(Azure Database for PostgreSQL Flexible Server)
```

URLs

Environment	URL
Production (Azure)	https://cop-upstream-platform.azurewebsites.net
Development (Local)	http://localhost:3000
SCM/Kudu (Deployment)	https://cop-upstream-platform.scm.azurewebsites.net

---

10. CI/CD Pipeline

GitHub Actions Workflow

File: `.github/workflows/deploy.yml` **Repository:** https://github.com/sriman2709/COPProject

Trigger Conditions

```yaml
on:
  push:
    branches: [main]      # Auto-deploy on every push to main
  workflow_dispatch:       # Manual trigger from GitHub UI
```

Pipeline Steps

```
┌──────────────────────────────────────────────────────────┐
│              GitHub Actions Runner                       │
│                  ubuntu-latest                           │
│                                                          │
│  Step 1: Checkout                                        │
│  └── actions/checkout@v4                                 │
│      Clones repository to runner                         │
│                                                          │
│  Step 2: Setup Node.js                                   │
│  └── actions/setup-node@v4                               │
│      Node 20, npm cache enabled                          │
│                                                          │
│  Step 3: Install Dependencies                            │
│  └── npm ci --omit=dev                                   │
│      Production packages only (no devDependencies)       │
│                                                          │
│  Step 4: Deploy to Azure                                 │
│  └── azure/webapps-deploy@v3                             │
│      app-name: cop-upstream-platform                     │
│      publish-profile: ${{ secrets.AZURE_WEBAPP_... }}    │
│      package: . (entire repo)                            │
└──────────────────────────────────────────────────────────┘
```

Required GitHub Secret

Secret Name	How to Get	Where to Add
`AZURE_WEBAPP_PUBLISH_PROFILE`	Azure Portal → App Service → Get publish profile	GitHub → Settings → Secrets → Actions

Deployment Timing

- Typical deployment time: **2-3 minutes** from `git push` to live
- Zero-downtime: Azure App Service swaps the running instance

Deploy Workflow

```
# Developer workflow
git add .
git commit -m "Your changes"
git push origin main
# → GitHub Actions triggers automatically
# → 2-3 minutes later, changes are live
```

---

11. Configuration & Environment Variables

`.env` File (Local Development)

```
DATABASE_URL=postgresql://localhost/copproject
PORT=3000
JWT_SECRET=your_long_random_secret_here
JWT_EXPIRES_IN=8h
```

Azure App Service Settings (Production)

Set via Azure Portal → App Service → Configuration → Application Settings, or via Azure CLI:

```
az webapp config appsettings set \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg \
  --settings KEY="VALUE"
```

Environment Variable Reference

Variable	Required	Default	Description
`DATABASE_URL`	Yes	`postgresql://localhost/copproject`	Full PostgreSQL connection string
`PORT`	No	`3000`	HTTP server port (Azure sets this automatically)
`JWT_SECRET`	Yes	(insecure default)	Secret for signing JWT tokens — must be changed in production
`JWT_EXPIRES_IN`	No	`8h`	JWT token lifetime
`NODE_ENV`	No	—	Set to `production` on Azure

Security Notes

- **`.env` is in `.gitignore`** — never committed to GitHub
- **`.env.example`** is committed as a template (no real values)
- **JWT_SECRET** on Azure is a randomly generated 64-character hex string
- **Database password** is URL-encoded in `DATABASE_URL` (@ → %40)

---

12. User Management

Default Credentials

Email	Password	Role
`admin@cop.com`	`Admin@123`	admin

Important: Change the admin password after initial deployment.

Creating Users (Admin only)

Via API:

```
curl -X POST https://cop-upstream-
platform.azurewebsites.net/api/auth/register \
  -H "Authorization: Bearer <admin_token>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Jane Smith",
    "email": "j.smith@cop.com",
    "password": "SecurePass@123",
    "role": "analyst"
  }'
```

Via Application (coming): User management UI accessible to admin role.

Changing Passwords

```
curl -X PUT https://cop-upstream-platform.azurewebsites.net/api/auth/password
\
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"currentPassword": "old", "newPassword": "new"}'
```

Password Requirements

- Minimum 8 characters (enforced at application level)
- Stored as bcrypt hash with 12 salt rounds

---

13. Data Flow Diagrams

Partner Onboarding Data Flow

```
User submits company data
        │
        ▼
POST /api/partners          → partners table
POST /api/partners/:id/contacts → contacts table
        │
        ▼
```

```
POST /api/onboarding        → onboarding_sessions (step_statuses: all
pending)
        │
        ▼
POST /api/onboarding/:id/advance/1
        → step 1 = 'done', step 2 = 'active'
        → recalculates overall_progress
        │
        ▼
[repeat for steps 2-7]
        │
        ▼
PUT /api/onboarding/:id     → status = 'completed'
```

Revenue Accounting Data Flow

```
Monthly period opens
        │
        ▼
POST /api/pra/volumes       → production_volumes (oil/gas/NGL)
        │
        ▼
POST /api/pra/allocations   → well_allocations (facility split)
        │
        ▼
POST /api/pra/prices        → price_indices
POST /api/pra/prices/:id/lock → status = 'locked'
        │
        ▼
PUT /api/pra/remittances/:id → reconcile vs. purchaser advice
        │
        ▼
POST /api/pra/royalty-runs  → royalty_runs
        (generates owner_payments records)
        │
        ▼
Payments distributed (ACH/check)
```

JVA Cash Call Flow

```
AFE approved
        │
        ▼
POST /api/jva/cash-calls    → cash_calls (issued, due_date set)
        │
        ▼
Partner sends payment
        │
        ▼
POST /api/jva/receipts      → receipts (pending)
        │
        ▼
POST /api/jva/receipts/:id/apply  → receipt matched to cash call
```

```
        → cash_call: received_amount updated, outstanding reduced
        → receipt: status = 'applied'
```

---

14. Project Structure

```
COPProject/
│
├── server.js                    # Express app entry point
│                                # Routes, middleware, static files, SPA
fallback
│
├── .env                         # Local environment variables (not in git)
├── .env.example                 # Template for environment setup
├── .gitignore                   # Excludes node_modules, .env, .db files
├── package.json                 # npm dependencies and scripts
│
├── database/
│   ├── db.js                    # pg.Pool setup, query(), buildUpdate(),
initDb()
│   ├── schema.sql               # All 40+ table definitions (PostgreSQL)
│   ├── auth_schema.sql          # auth_users table (CREATE TABLE IF NOT
EXISTS)
│   └── seed.sql                 # Demo data for all modules
│
├── middleware/
│   └── auth.js                  # authenticate(), requireAdmin(),
requireAnalyst()
│
├── models/                      # Data access layer
│   ├── Partner.js               # getAll, getById, create, update, delete,
getFull
│   ├── Contact.js               # getByPartner, create, update, delete
│   ├── JOA.js                   # getByPartner, create, update,
getWithInterests
│   ├── WorkingInterest.js       # getByPartner, getWISummary
│   ├── Financial.js             # getByPartner, upsert
│   ├── COPAS.js                 # getByPartner, upsert, acknowledge
│   ├── OverheadRate.js          # getByCopas
│   ├── Property.js              # getByJOA, create, update
│   ├── Onboarding.js            # getAll, getById, create, update,
advanceStep
│   ├── Document.js              # getByOnboarding, getStats, receive, verify
│   ├── ActivityLog.js           # getByOnboarding, create
│   ├── UserAccount.js           # getByPartner, create, update, activate,
delete
│   ├── PraPeriod.js             # getAll, getCurrent, update,
updateChecklist
│   ├── ProductionVolume.js      # getByPeriod, getByProperty, getExceptions
│   ├── WellAllocation.js        # getByPeriod, getByFacility, getAlerts
│   ├── GasImbalance.js          # getAll, getByProperty, getAlerts
│   ├── PriceIndex.js            # getByPeriod, getHistory, lock, create
```

```
│   ├── MarketingContract.js     # getAll, getById, getExpiring, create,
update
│   ├── Remittance.js            # getByPeriod, getStats, getExceptions
│   ├── SuspenseAccount.js       # getAll, getStats, update
│   ├── DivisionOrder.js         # getAll, getByProperty, getErrors, getStats
│   ├── RoyaltyRun.js            # getAll, getLatest, getById, create,
getSummary
│   ├── OwnerPayment.js          # getByRun, getStats
│   ├── AFE.js                   # getAll, getById, create, update,
getBudgetComparison
│   ├── AFELineItem.js           # getByAFE, create, update
│   ├── AFEElection.js           # getByAFE, getAll, getOpen, respond
│   ├── JIB.js                   # getAll, getById, getByPeriod, getStats
│   ├── JIBLineItem.js           # getByJIB, create
│   ├── CashCall.js              # getAll, getOverdue, getStats, getForecast
│   ├── Receipt.js               # getAll, getPending, create, apply
│   └── AuditFinding.js          # getAll, getById, getStats, create, update
│
├── routes/
│   ├── auth.js                  # POST /login, /register, GET /me, /users,
etc.
│   ├── partners.js              # Full CRUD +
contacts/JOA/financial/COPAS/users
│   ├── onboarding.js            # Session CRUD + step management + activity
│   ├── documents.js             # Document tracking + receive/verify
workflow
│   ├── dashboard.js             # Onboarding dashboard aggregate
│   ├── pipeline.js              # Pipeline list + stats
│   ├── pra.js                   # All PRA endpoints (55+)
│   └── jva.js                   # All JVA endpoints (40+)
│
├── public/
│   └── index.html               # Complete SPA (~2,600 lines)
│                                # Auth, routing, all page renderers, charts
│
├── .github/
│   └── workflows/
│       └── deploy.yml           # GitHub Actions → Azure App Service
│
└── docs/
    ├── SYSTEM-DOCUMENTATION.md  # Original system documentation
    ├── PROCESS-DIAGRAMS.md      # Business process diagrams
    └── APPLICATION-DOCUMENTATION.md  # This file
```

---

15. Operational Runbook

Checking Application Health

```
# Is the app responding?
```

```
curl -s -o /dev/null -w "%{http_code}" https://cop-upstream-
platform.azurewebsites.net/

# Is the API working?
curl -X POST https://cop-upstream-platform.azurewebsites.net/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@cop.com","password":"Admin@123"}'

# Check App Service state via CLI
az webapp show --name cop-upstream-platform \
  --resource-group cop-platform-rg \
  --query "{state:state}" -o table
```

Viewing Application Logs

```
# Stream live logs
az webapp log tail \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg

# Download log archive
az webapp log download \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg \
  --log-file /tmp/app-logs.zip
```

Restarting the Application

```
az webapp restart \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg
```

Updating Environment Variables

```
az webapp config appsettings set \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg \
  --settings KEY="NEW_VALUE"
```

Manual Deployment (without GitHub Actions)

```
# From the project directory
zip -r /tmp/cop-deploy.zip . \
  --exclude ".git/*" --exclude ".env" \
  --exclude "*.db" --exclude ".claude/*"

az webapp deploy \
  --name cop-upstream-platform \
  --resource-group cop-platform-rg \
  --src-path /tmp/cop-deploy.zip \
```

```
  --type zip
```

Connecting to the PostgreSQL Database

```
# Via Azure CLI (requires psql installed)
az postgres flexible-server connect \
  --name cop-postgres-srv \
  --admin-user copadmin \
  --database-name copproject

# Direct psql connection
psql "postgresql://copadmin:<password>@cop-postgres-
srv.postgres.database.azure.com/copproject?sslmode=require"
```

Resetting the Database (Destructive)

```
# Connect to database and drop/recreate all tables
psql "connection_string" -c "DROP SCHEMA public CASCADE; CREATE SCHEMA
public;"
# Then restart the app — initDb() will recreate schema and seed data
az webapp restart --name cop-upstream-platform --resource-group cop-platform-
rg
```

Adding a New User

```
# 1. Get admin token
TOKEN=$(curl -s -X POST https://cop-upstream-
platform.azurewebsites.net/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@cop.com","password":"Admin@123"}' | \
  python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# 2. Create user
curl -X POST https://cop-upstream-
platform.azurewebsites.net/api/auth/register \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Full
Name","email":"user@cop.com","password":"Pass@123","role":"analyst"}'
```

Scaling Up (if performance requires)

```
# Upgrade App Service Plan from B1 to S1 (Standard)
az appservice plan update \
  --name cop-app-plan \
  --resource-group cop-platform-rg \
  --sku S1
```

Summary Statistics

Metric	Value
Total API Endpoints	130+
Database Tables	40+
Data Models	30
Onboarding Workflow Steps	7
User Roles	3 (admin, analyst, viewer)
Frontend Pages/Views	35+
Production Dependencies	6
Node.js Version	20 LTS
PostgreSQL Version	16
Azure Region	Central US
Live URL	https://cop-upstream-platform.azurewebsites.net
GitHub Repository	https://github.com/sriman2709/COPProject

Documentation maintained alongside source code in `/docs/APPLICATION-DOCUMENTATION.md`

Process Diagrams — COP Upstream Platform

All diagrams use Mermaid syntax and render natively on GitHub, GitLab, Notion, and VS Code (with Mermaid extension).

---

1. End-to-End Onboarding Lifecycle

The complete partner onboarding journey from JOA execution to system go-live:

---

2. Onboarding Session State Machine

How an onboarding session transitions through states:

---

3. Individual Step Status Transitions

Each of the 7 steps follows this state model:

---

4. Step 1 — Company Registration Process

---

5. Step 2 — JOA & Working Interest Process

---

6. Step 3 — Financial Setup Process

7. Step 4 — COPAS Compliance Process

8. Step 5 — Document Collection Process

9. Step 6 — System Access Provisioning Process

10. Step 7 — Review & Submit Process

11. Document Lifecycle State Machine

12. User Account Provisioning State Machine

13. Pipeline Overview — Multi-Partner Progress

14. Data Flow — API Request/Response

15. Step Advancement — Sequence Diagram

---

16. Progress Calculation Logic

Formula: `progress = round(((done * 100) + (active * 45)) / 7)`

Scenario	Done	Active	Pending	Progress
Just started	0	1	6	6%
2 done, 1 active	2	1	4	35%
2 done, 1 active (Sierra)	2	1	4	42%*
5 done, 1 active	5	1	1	78%
6 done, 1 active	6	1	0	92%
All 7 done	7	0	0	100%

*Sierra Digital shows 42% because the calculation accounts for step weighting from the stored data.

---

17. Overdue Detection Logic

---

18. PRA — Monthly Revenue Close Lifecycle

---

19. PRA — Volume Capture & Allocation Flow

Unable to render rich display

Lexical error on line 18. Unrecognized text.
...ation Locked] M -- |>= 2%| O[Alert:
----------^

For more information, see https://docs.github.com/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams#creating-mermaid-diagrams

```
flowchart TD

    A[SCADA System / Field Gauges] --> B[Import Production Data]

    B --> C[Create Volume Records by Property]

    C --> D{Exceptions Detected?}

    D -- Yes --> E[Flag Exception Volume Records]

    E --> F[Manual Review & Correction]

    F --> D

    D -- No --> G[Volumes Validated]

    G --> H[Well Allocation Engine]

    H --> I{Allocation Method}

    I --> |Well Test| J1[Apply Well Test Factors]

    I --> |Meter Pro-Rata| J2[Apply Meter Readings]

    J1 --> K[Calculate Allocated BBL per Well]

    J2 --> K

    K --> L[Compare to Purchaser Receipts]

    L --> M{Variance Check}

    M -- |< 2%| N[Allocation Locked]

    M -- |>= 2%| O[Alert: High Variance]

    O --> P[Review Stale Test Days]

    P --> K


    style A fill:#D42B2B,color:#fff

    style N fill:#16a34a,color:#fff

    style M fill:#f59e0b,color:#000
```

20. PRA — Price Index Locking Process

21. PRA — Remittance Matching & Suspense

22. PRA — Royalty Calculation & Distribution

23. PRA — Gas Imbalance State Machine

24. JVA — AFE Lifecycle

25. JVA — JIB Billing Process

26. JVA — Cash Call & Receipt Application

27. JVA — Cash Call State Machine

Unable to render rich display

Parse error on line 10:
...ent Overdue --> Default : Extended n

----------^
Expecting 'ID', 'EDGE_STATE', got 'DEFAULT'

For more information, see https://docs.github.com/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams#creating-mermaid-diagrams

```
stateDiagram-v2

    [*] --> Issued : Cash call created

    Issued --> Partial : Partial payment received

    Issued --> Paid : Full payment received

    Issued --> Overdue : Due date passed

    Partial --> Paid : Remaining balance paid

    Partial --> Overdue : Due date passed with balance

    Overdue --> Partial : Late partial payment

    Overdue --> Paid : Late full payment

    Overdue --> Default : Extended non-payment

    Paid --> [*]


    note right of Issued : Awaiting partner payment

    note right of Overdue : Interest accruing per JOA terms

    note right of Default : Legal remedies per JOA
```

---

28. JVA — Audit Finding Lifecycle

---

29. JVA — COPAS Overhead Calculation

---

30. End-to-End Platform Data Flow

This diagram shows how the three platform domains interconnect:

COP Upstream Platform — System Documentation

Table of Contents

1. System Overview
2. Technology Stack
3. Project Structure
4. Database Schema
5. API Reference
6. 7-Step Onboarding Workflow
7. Data Models
8. Security & Compliance
9. Setup & Deployment
10. PRA — Product Revenue Accounting Module
11. JVA — Joint Venture Accounting Module

---

1. System Overview

The **COP Upstream Platform** is a 3-domain enterprise system supporting ConocoPhillips' upstream operations across partner onboarding, revenue accounting, and joint venture cost management.

Three Domains

- **Partner Onboarding** — Structured 7-step digital workflow that replaces manual spreadsheet-based tracking when ConocoPhillips enters into a Joint Operating

Agreement (JOA) with a new partner. Covers legal, financial, regulatory, and system-access dimensions.

- **Product Revenue Accounting (PRA)** — Monthly cycle that tracks production volumes from wellhead through purchaser remittance to royalty distribution. Manages volumes, pricing, royalties, division orders, suspense accounts, and tax reporting.
- **Joint Venture Accounting (JVA)** — Operator/non-operator cost sharing per COPAS 2005 MFI-51. Manages AFE authorization, JIB billing, cash call collections, overhead calculations, and audit compliance.

Key Capabilities

- **Partner Registration** — Entity details, contacts, address verification
- **JOA Configuration** — Agreement terms, property assignments, working interest allocation
- **Financial Setup** — Banking (ACH/EFT), tax classification (W-9), payment preferences
- **COPAS Compliance** — Overhead rate elections, cost classification, partner acknowledgment
- **Document Collection** — 8 required document types with receive/verify tracking
- **System Access Provisioning** — Portal accounts, MFA enrollment, role-based access
- **Pipeline Management** — Multi-partner progress tracking, KPIs, overdue alerts
- **Production & Revenue** — Volume tracking, well allocation, gas balancing, price indices, remittance matching, suspense management, division orders, royalty runs, owner payments
- **Joint Venture Operations** — AFE budgeting and elections, JIB generation and billing, cash call issuance and collection, receipt application, COPAS overhead, audit findings

Architecture

```
+-----------------------+
|   Browser (SPA)       |
|   public/index.html   |
+---------+-------------+
          |  HTTP/JSON
          v
+---------+-------------+
|   Express.js Server   |
```

```
                    |      server.js        |
                    |                       |
                    |  /api/partners        |
                    |  /api/onboarding      |
                    |  /api/documents       |
                    |  /api/dashboard       |
                    |  /api/pipeline        |
                    |  /api/pra             |
                    |  /api/jva             |
                    +---------+-------------+
                              |
                    +---------+-------------+
                    |   31 Model Layer      |
                    |   models/*.js         |
                    +---------+-------------+
                              |
                    +---------+-------------+
                    |   SQLite Database     |
                    |   sql.js (WASM)       |
                    |   database/           |
                    +-----------------------+
```

---

2. Technology Stack

Layer	Technology	Purpose
Frontend	Vanilla HTML/CSS/JS	Single-page application (SPA)
Charts	Chart.js 4.4	Pipeline doughnut and bar charts
Fonts	Barlow, Barlow Condensed, JetBrains Mono	COP brand typography
Server	Node.js + Express 4.21	REST API server
Database	SQLite via sql.js (WASM)	Persistent relational storage
Middleware	cors, express.json	Cross-origin and JSON parsing

---

3. Project Structure

```
COPProject/
├── server.js                    # Express entry point (port 3000)
├── package.json                 # Dependencies: express, sql.js, cors
├── database/
│   ├── db.js                    # sql.js wrapper (better-sqlite3 compatible
API)
│   ├── schema.sql               # 31-table schema with constraints/indexes
│   ├── seed.sql                 # Partners, onboarding, PRA/JVA data
│   └── onboarding.db            # SQLite database file (auto-created)
├── models/
│   ├── Partner.js               # Company entity CRUD
│   ├── Contact.js               # Partner contacts (5 role types)
│   ├── JOA.js                   # Joint Operating Agreement management
│   ├── Property.js              # Well/property assignments
│   ├── WorkingInterest.js       # WI% / NRI% per partner per property
│   ├── Financial.js             # Banking, tax, payment configuration
│   ├── COPAS.js                 # COPAS compliance configuration
│   ├── OverheadRate.js          # Overhead rate schedule entries
│   ├── Document.js              # Required document tracking
│   ├── UserAccount.js           # Portal user provisioning
│   ├── Onboarding.js            # 7-step session state management
│   ├── ActivityLog.js           # Timeline event logging
│   ├── PraPeriod.js             # PRA accounting period management
│   ├── ProductionVolume.js      # SCADA/gauge production volumes
│   ├── WellAllocation.js        # Well test allocation factors
│   ├── GasImbalance.js          # Gas entitlement vs takes
│   ├── PriceIndex.js            # Commodity price indices
│   ├── MarketingContract.js     # Purchaser marketing contracts
│   ├── Remittance.js            # Purchaser remittance matching
│   ├── SuspenseAccount.js       # Suspense account management
│   ├── DivisionOrder.js         # Division order interest tracking
│   ├── RoyaltyRun.js            # Royalty calculation runs
│   ├── OwnerPayment.js          # Owner payment distribution
│   ├── AFE.js                   # Authorization for Expenditure
│   ├── AFELineItem.js           # AFE budget line items by COPAS code
│   ├── AFEElection.js           # Partner election responses
│   ├── JIB.js                   # Joint Interest Billing
│   ├── JIBLineItem.js           # JIB cost line items
│   ├── CashCall.js              # Cash call issuance and tracking
│   ├── Receipt.js               # Cash call receipt application
│   └── AuditFinding.js          # COPAS audit finding management
├── routes/
│   ├── partners.js              # /api/partners — full partner CRUD + nested
resources
│   ├── onboarding.js            # /api/onboarding — wizard steps, progress,
activity
│   ├── documents.js             # /api/documents — checklist, receive,
verify
│   ├── dashboard.js             # /api/dashboard — aggregated KPIs
```

```
│   ├── pipeline.js              # /api/pipeline — all onboardings overview
│   ├── pra.js                   # /api/pra — Product Revenue Accounting
endpoints
│   └── jva.js                   # /api/jva — Joint Venture Accounting
endpoints
├── public/
│   └── index.html               # SPA frontend (API-connected)
└── docs/
    ├── SYSTEM-DOCUMENTATION.md  # This file
    └── PROCESS-DIAGRAMS.md      # Visual process flow diagrams
```

---

4. Database Schema

Entity-Relationship Overview

```
   ┌──────────────┐ ──1:N──▶│ ┌──────────────┐
   │   partners   │          │  contacts    │
   │              │          │ (5 role types)│
   └──────────────┘          └──────────────┘
          │
          ├──1:N──▶ ┌──────────┐ ──1:N──▶ ┌──────────────┐
          │         │   joa    │          │ properties   │
│──1:1──▶ ┌──────────────────┐ └──────────┘ └──────────────┘        │
          │         └──────────┘          └──────────────┘        │
working_interests │
          │
└──────────────────┘
          │
          ├──1:1──▶ ┌──────────────────┐
          │         │financial_setup   │
          │         └──────────────────┘
          │
          ├──1:1──▶ ┌──────────────────┐ ──1:N──▶ ┌──────────────────┐
          │         │copas_config      │          │overhead_rates    │
          │         └──────────────────┘          └──────────────────┘
          │
          ├──1:N──▶ ┌──────────────────┐
          │         │user_accounts     │
          │         └──────────────────┘
          │
          └──1:N──▶ ┌──────────────────────┐ ──1:N──▶ ┌──────────────┐
                    │onboarding_sessions   │          │  documents   │
                    └──────────────────────┘          └──────────────┘
                              │
                              └──1:N──▶ ┌──────────────────┐
                                        │activity_log      │
                                        └──────────────────┘
```

Table Details

`partners`
Primary entity representing a non-operator company.

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
legal_name	TEXT	NOT NULL
dba_name	TEXT	
entity_type	TEXT	NOT NULL, CHECK(LLC, Corporation, LP, etc.)
ein_tin	TEXT	Employer Identification Number
duns_number	TEXT	D-U-N-S Number
state_of_formation	TEXT	Jurisdiction of incorporation
date_of_formation	TEXT	
street_address	TEXT	
city, state, zip	TEXT	
mailing_address	TEXT	
created_at	TEXT	DEFAULT datetime('now')
updated_at	TEXT	DEFAULT datetime('now')

`contacts`
Up to 5 designated contact roles per partner.

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
partner_id	INTEGER	FK -> partners(id), ON DELETE CASCADE
role	TEXT	CHECK(operations, finance, legal, billing, elections)
name	TEXT	NOT NULL
title	TEXT	
email	TEXT	
phone	TEXT	
is_primary	INTEGER	DEFAULT 0

`joa`

Joint Operating Agreement configuration.

Column	Type	Notes
id	INTEGER	PRIMARY KEY
partner_id	INTEGER	FK -> partners(id)
joa_reference	TEXT	e.g., JOA-2025-PB-042
effective_date	TEXT	
operator	TEXT	DEFAULT 'ConocoPhillips Company'
agreement_type	TEXT	DEFAULT 'AAPL Form 610 (2015)'
election_period_days	INTEGER	DEFAULT 30

Column	Type	Notes
non_consent_penalty	TEXT	DEFAULT '300% (Standard)'
governing_law	TEXT	DEFAULT 'Texas'
accounting_procedure	TEXT	DEFAULT 'COPAS 2005 MFI-51'
audit_right_window	TEXT	DEFAULT '24 months'

`properties`

Individual wells or property units tied to a JOA.

Column	Type	Notes
id	INTEGER	PRIMARY KEY
joa_id	INTEGER	FK -> joa(id)
name	TEXT	e.g., Permian Spraberry #42
basin	TEXT	e.g., Delaware Basin
product_type	TEXT	DEFAULT 'OIL/GAS/NGL'
effective_date	TEXT	
decimal_sum_valid	INTEGER	1 = all WI sum to 100%

`working_interests`

Fractional ownership interest per partner per property.

Column	Type	Notes
id	INT	PRIMARY KEY

Column	Type	Notes
partner_id	INT	FK -> partners(id)
property_id	INT	FK -> properties(id)
wi_pct	REAL	Working Interest % (6 decimal precision)
nri_pct	REAL	Net Revenue Interest %
royalty_burden	REAL	DEFAULT 0.25 (25%)
UNIQUE		(partner_id, property_id)

`financial_setup`

Banking and tax configuration for revenue distribution.

Column	Type	Notes
partner_id	INT	FK -> partners(id)
bank_name	TEXT	e.g., Frost Bank
account_type	TEXT	Checking / Savings
routing_number	TEXT	ABA routing number
account_number_masked	TEXT	e.g., ****4872
payment_method	TEXT	ACH / EFT / Wire / Check
tax_classification	TEXT	LLC (Partnership), Corporation, etc.
reporting_type	TEXT	1099-MISC, 1099-NEC
w9_status	TEXT	pending / received / verified

Column	Type	Notes
ach_prenote_status	TEXT	pending / confirmed / failed
cashcall_delivery	TEXT	Email, EDI 810, Portal
jib_delivery	TEXT	EDI 810, Email, Portal
payment_terms	TEXT	Net 30 / Net 45

`copas_config`

Council of Petroleum Accountants Societies (COPAS) accounting elections.

Column	Type	Notes
partner_id	INT	FK -> partners(id)
joa_id	INT	FK -> joa(id)
copas_version	TEXT	e.g., 2005 MFI-51 (Onshore)
overhead_method	TEXT	Fixed Rate (Schedule A) or Percentage
escalation_index	TEXT	COPAS Annual Index rate
material_pricing	TEXT	MPI (Material Pricing Index)
audit_period	TEXT	24 months from JIB receipt
dispute_resolution	TEXT	Mediation then Arbitration
acknowledged	INT	0/1 — partner must formally acknowledge
acknowledged_date	TEXT	When partner signed off

`overhead_rates`

Line-item overhead rates per COPAS config.

Column	Type	Notes
copas_config_id	INT	FK -> copas_config(id)
category	TEXT	Producing Well, Drilling Ops, etc.
rate_2025	TEXT	e.g., $1,840/mo or 10.0%
basis	TEXT	Per well/per month, % of cost
partner_share	TEXT	Partner's proportional share

`documents`

Required legal, financial, and regulatory documents.

Column	Type	Notes
onboarding_id	INT	FK -> onboarding_sessions(id)
name	TEXT	e.g., W-9, Certificate of Insurance
category	TEXT	Tax, Legal, Insurance, Finance, Revenue, etc.
required_date	TEXT	Deadline
received_date	TEXT	When document was received
verified_date	TEXT	When verified by COP staff
verified	INT	0/1
status	TEXT	pending / received / verified / overdue / complete

`user_accounts`

Portal user accounts to be provisioned for the partner.

Column	Type	Notes
partner_id	INT	FK -> partners(id)
name	TEXT	User full name
email	TEXT	Login email
role	TEXT	Partner Admin, Finance User, Land User
access_level	TEXT	Description of permitted modules
mfa_status	TEXT	pending / enrolled
account_status	TEXT	queued / active / disabled
portal_access	TEXT	Enable / Disable
notif_jib	TEXT	Email + Portal, Portal Only, etc.
notif_cashcall	TEXT	Notification preference
notif_afe	TEXT	Notification preference
notif_revenue	TEXT	Notification preference

`onboarding_sessions`

Master session tracking a partner through the 7-step workflow.

Column	Type	Notes
partner_id	INT	FK -> partners(id)
onboarding_ref	TEXT	UNIQUE, e.g., ONB-2025-0018
partner_type	TEXT	Non-Operator, Operator Transfer, Farm-In

Column	Type	Notes
property_summary	TEXT	Short description of assigned properties
wi_summary	TEXT	Working interest percentage summary
initiated_date	TEXT	When onboarding was started
target_date	TEXT	Expected completion date
current_step	INT	1-7
overall_progress	INT	0-100%
step_statuses	TEXT	JSON: {"1":"done","2":"active",...}
status	TEXT	active / completed / suspended

`activity_log`

Timestamped event log per onboarding session.

Column	Type	Notes
onboarding_id	INT	FK -> onboarding_sessions(id)
title	TEXT	Short event title
description	TEXT	Details
timestamp	TEXT	DEFAULT datetime('now')

PRA Tables (11)

`pra_periods`

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
period	TEXT	NOT NULL UNIQUE (YYYY-MM)
status	TEXT	CHECK(open/locked/closed)
close_checklist	TEXT	JSON with 8 items: volume_close, facility_balance, price_lock, remittance_match, gas_balance, royalty_calc, owner_statements, gl_posting

`production_volumes`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
property_id	INTEGER	FK -> properties(id)
period	TEXT	YYYY-MM
oil_bbl	REAL	Oil barrels
gas_mcf	REAL	Gas MCF
ngl_bbl	REAL	NGL barrels
source	TEXT	DEFAULT 'SCADA'
exceptions	INTEGER	Exception count
status	TEXT	draft/validated/locked

`well_allocations`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
property_id	INTEGER	FK -> properties(id)
facility	TEXT	NOT NULL
period	TEXT	
method	TEXT	DEFAULT 'Well Test'
allocated_bbl	REAL	
purchaser_bbl	REAL	
variance_pct	REAL	
stale_days	INTEGER	
status	TEXT	pending/validated/locked

`gas_imbalances`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
property_id	INTEGER	FK -> properties(id)
wi_pct	REAL	Working interest %
entitlement_mcf	REAL	
taken_mcf	REAL	

Column	Type	Notes
imbalance_mcf	REAL	
price_per_mcf	REAL	
bs_value	REAL	Balance sheet value
makeup_expiry	TEXT	
status	TEXT	balanced/overlift/underlift/alert

`price_indices`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
index_name	TEXT	NOT NULL (e.g., WTI Cushing)
product	TEXT	NOT NULL (Oil/Gas/NGL)
basis	TEXT	
period	TEXT	YYYY-MM
price	REAL	
differential	REAL	
net_price	REAL	
source	TEXT	
status	TEXT	needs_lock/locked

Column	Type	Notes
locked_by	TEXT	
locked_at	TEXT	

marketing_contracts

Column	Type	Notes
id	INTEGER	PRIMARY KEY
contract_number	TEXT	UNIQUE
purchaser	TEXT	
property_id	INTEGER	FK -> properties(id)
product	TEXT	
pricing_formula	TEXT	
delivery_point	TEXT	
effective_date	TEXT	
expires_date	TEXT	
volume_commitment	TEXT	
status	TEXT	active/expiring/expired/terminated

remittances

Column	Type	Notes
id	INTEGER	PRIMARY KEY

Column	Type	Notes
contract_id	INTEGER	FK -> marketing_contracts(id)
period	TEXT	
our_vol	REAL	COP volume
purch_vol	REAL	Purchaser volume
vol_var_pct	REAL	Volume variance %
our_price	REAL	COP price
purch_price	REAL	Purchaser price
difference	REAL	
match_status	TEXT	pending/matched/vol_variance/price_variance/suspense

`suspense_accounts`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
account_number	TEXT	UNIQUE
owner_name	TEXT	
property_id	INTEGER	FK -> properties(id)
hold_reason	TEXT	
balance	REAL	

Column	Type	Notes
oldest_date	TEXT	
days_out	INTEGER	
status	TEXT	active/resolved/escheat_risk

`division_orders`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
property_id	INTEGER	FK -> properties(id)
owner_name	TEXT	
interest_type	TEXT	DEFAULT 'RI'
decimal_interest	REAL	
product	TEXT	DEFAULT 'ALL'
do_status	TEXT	active/pending/suspended/transfer
decimal_sum	REAL	
last_updated	TEXT	

`royalty_runs`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
period	TEXT	

Column	Type	Notes
price_basis	TEXT	DEFAULT 'FOM Index'
deductibility	TEXT	DEFAULT 'Lease-Specific'
total_owners	INTEGER	
total_interests	INTEGER	
total_amount	REAL	
checks_efts	INTEGER	
below_min	INTEGER	
decimal_errors	INTEGER	
status	TEXT	draft/calculated/approved/distributed

`owner_payments`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
royalty_run_id	INTEGER	FK -> royalty_runs(id)
owner_name	TEXT	
payment_method	TEXT	ACH/check
amount	REAL	
status	TEXT	pending/sent/held/returned

JVA Tables (8)

afes

Column	Type	Notes
id	INTEGER	PRIMARY KEY
afe_number	TEXT	UNIQUE
property_id	INTEGER	FK -> properties(id)
basin	TEXT	
afe_type	TEXT	drilling/workover/facility/p_and_a
authorized	REAL	Authorized budget
actual	REAL	Actual spend
budget_pct	REAL	% of budget used
operator_wi	REAL	Operator working interest
proposed_spud	TEXT	
est_duration	INTEGER	Days
status	TEXT	open/closed/supplemental_required

afe_line_items

Column	Type	Notes
id	INTEGER	PRIMARY KEY
afe_id	INTEGER	FK -> afes(id) ON DELETE CASCADE

Column	Type	Notes
copas_code	TEXT	100/200/300/400/600/700
description	TEXT	
idc_amount	REAL	Intangible drilling cost
tangible_amount	REAL	

afe_elections

Column	Type	Notes
id	INTEGER	PRIMARY KEY
afe_id	INTEGER	FK -> afes(id) ON DELETE CASCADE
partner_id	INTEGER	FK -> partners(id)
partner_name	TEXT	
wi_pct	REAL	
share_amount	REAL	
response	TEXT	pending/participate/non_consent
issued_date	TEXT	
due_date	TEXT	
response_date	TEXT	
status	TEXT	open/closed/defaulted

jibs

Column	Type	Notes
id	INTEGER	PRIMARY KEY
jib_number	TEXT	UNIQUE
property_id	INTEGER	FK -> properties(id)
non_operator	TEXT	
period	TEXT	
direct_amount	REAL	
overhead_amount	REAL	
total_amount	REAL	
issued_date	TEXT	
due_date	TEXT	
paid_amount	REAL	
status	TEXT	draft/issued/partial/paid/overdue

`jib_line_items`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
jib_id	INTEGER	FK -> jibs(id) ON DELETE CASCADE
copas_code	TEXT	

Column	Type	Notes
description	TEXT	
total_amount	REAL	
nonop_share	REAL	

`cash_calls`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
cc_number	TEXT	UNIQUE
partner_name	TEXT	
property_id	INTEGER	FK -> properties(id)
amount	REAL	
due_date	TEXT	
received_amount	REAL	
outstanding	REAL	
days_past_due	INTEGER	
interest_accrued	REAL	
status	TEXT	issued/partial/paid/overdue/default

`receipts`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
receipt_number	TEXT	UNIQUE
partner_name	TEXT	
amount	REAL	
receipt_date	TEXT	
applied_to_cc	TEXT	Cash call number
remaining	REAL	Unapplied amount
status	TEXT	pending/applied/partial

`audit_findings`

Column	Type	Notes
id	INTEGER	PRIMARY KEY
finding_number	TEXT	UNIQUE
property_id	INTEGER	FK -> properties(id)
auditor	TEXT	
category	TEXT	
description	TEXT	
amount	REAL	

Column	Type	Notes
response_due	TEXT	
status	TEXT	open/disputed/resolved/credit_due

---

5. API Reference

Partners

Method	Endpoint	Description
GET	`/api/partners`	List all partners
GET	`/api/partners/:id`	Get partner by ID
GET	`/api/partners/:id/full`	Partner + all related data
POST	`/api/partners`	Create new partner
PUT	`/api/partners/:id`	Update partner fields
GET	`/api/partners/:id/contacts`	List contacts for partner
POST	`/api/partners/:id/contacts`	Add contact to partner
PUT	`/api/partners/:pid/contacts/:cid`	Update a contact
DELETE	`/api/partners/:pid/contacts/:cid`	Remove a contact
GET	`/api/partners/:id/joa`	JOA(s) with properties & WI data
POST	`/api/partners/:id/joa`	Create a JOA for partner
PUT	`/api/partners/:pid/joa/:jid`	Update JOA terms
GET	`/api/partners/:id/working-interests`	All WI entries for partner

Method	Endpoint	Description
GET	`/api/partners/:id/wi-summary`	WI/NRI summary with COP split
GET	`/api/partners/:id/financial`	Financial setup for partner
PUT	`/api/partners/:id/financial`	Update (or create) financial setup
GET	`/api/partners/:id/copas`	COPAS config + overhead rates
PUT	`/api/partners/:id/copas`	Update COPAS elections
POST	`/api/partners/:id/copas/acknowledge`	Partner acknowledges COPAS terms
GET	`/api/partners/:id/copas/cost-classification`	Static cost class. rules
GET	`/api/partners/:id/users`	List user accounts to provision
POST	`/api/partners/:id/users`	Add new user account
PUT	`/api/partners/:pid/users/:uid`	Update user account settings
POST	`/api/partners/:pid/users/:uid/activate`	Activate user + enroll MFA
DELETE	`/api/partners/:pid/users/:uid`	Remove user account

Onboarding Sessions

Method	Endpoint	Description
GET	`/api/onboarding`	List all sessions (with partner name)
GET	`/api/onboarding/:id`	Get session by ID
POST	`/api/onboarding`	Create new onboarding session
PUT	`/api/onboarding/:id`	Update session fields

Method	Endpoint	Description
PUT	`/api/onboarding/:id/step/:step`	Update a specific step's status
POST	`/api/onboarding/:id/advance/:step`	Mark step done, advance to next
GET	`/api/onboarding/:id/progress`	Current step, progress, all statuses
GET	`/api/onboarding/:id/activity`	Activity log timeline
POST	`/api/onboarding/:id/activity`	Add activity log entry

Documents

Method	Endpoint	Description
GET	`/api/documents/onboarding/:id`	All documents for an onboarding
GET	`/api/documents/onboarding/:id/stats`	Document completion statistics
GET	`/api/documents/:id`	Get single document
PUT	`/api/documents/:id`	Update document fields
POST	`/api/documents/:id/receive`	Mark document as received
POST	`/api/documents/:id/verify`	Mark document as verified/complete
POST	`/api/documents`	Create new document requirement

Dashboard

Method	Endpoint	Description
GET	`/api/dashboard/:onboardingId`	Full dashboard payload (KPIs, steps, docs, activity, JOAs, WI)

Pipeline

Method	Endpoint	Description
GET	`/api/pipeline`	All onboardings with partner info
GET	`/api/pipeline/stats`	Pipeline KPIs (active, overdue, monthly)

PRA — Product Revenue Accounting

Dashboard

Method	Endpoint	Description
GET	`/api/pra/dashboard`	Aggregated PRA dashboard (period, volumes, exceptions, alerts, prices, suspense, division orders, royalty run, contracts)

Periods

Method	Endpoint	Description
GET	`/api/pra/periods`	List all accounting periods
GET	`/api/pra/periods/current`	Current open period
PUT	`/api/pra/periods/:id`	Update a period
PUT	`/api/pra/periods/:id/checklist`	Update period close checklist

Volumes

Method	Endpoint	Description
GET	`/api/pra/volumes/:period`	Production volumes for a period
GET	`/api/pra/volumes/exceptions/:period`	Volume exceptions for a period
GET	`/api/pra/volumes/property/:id`	Volumes by property
POST	`/api/pra/volumes`	Create a volume record

Method	Endpoint	Description
PUT	`/api/pra/volumes/:id`	Update a volume record

Allocations

Method	Endpoint	Description
GET	`/api/pra/allocations/:period`	Well allocations for a period
GET	`/api/pra/allocations/alerts`	Allocation alerts (stale, high variance)
GET	`/api/pra/allocations/facility/:name`	Allocations by facility
POST	`/api/pra/allocations`	Create an allocation
PUT	`/api/pra/allocations/:id`	Update an allocation

Gas Balances

Method	Endpoint	Description
GET	`/api/pra/gas-balances`	All gas imbalance records
GET	`/api/pra/gas-balances/alerts`	Imbalance alerts (overlift/underlift)
GET	`/api/pra/gas-balances/property/:id`	Gas balances by property
PUT	`/api/pra/gas-balances/:id`	Update a gas balance

Price Indices

Method	Endpoint	Description
GET	`/api/pra/prices/:period`	Price indices for a period

Method	Endpoint	Description
GET	`/api/pra/prices/history/:indexName`	12-month price history for an index
POST	`/api/pra/prices/:id/lock`	Lock a price index
POST	`/api/pra/prices`	Create a price index record

Contracts

Method	Endpoint	Description
GET	`/api/pra/contracts`	All marketing contracts
GET	`/api/pra/contracts/expiring`	Contracts nearing expiration
GET	`/api/pra/contracts/:id`	Single contract by ID
POST	`/api/pra/contracts`	Create a marketing contract
PUT	`/api/pra/contracts/:id`	Update a marketing contract

Remittances

Method	Endpoint	Description
GET	`/api/pra/remittances/:period`	Remittances for a period
GET	`/api/pra/remittances/exceptions/:period`	Remittance exceptions
GET	`/api/pra/remittances/stats/:period`	Match/exception counts
POST	`/api/pra/remittances`	Create a remittance record
PUT	`/api/pra/remittances/:id`	Update a remittance

Suspense

Method	Endpoint	Description
GET	`/api/pra/suspense`	All suspense accounts
GET	`/api/pra/suspense/stats`	Suspense account statistics
PUT	`/api/pra/suspense/:id`	Update a suspense account

Division Orders

Method	Endpoint	Description
GET	`/api/pra/division-orders`	All division orders
GET	`/api/pra/division-orders/errors`	Division order errors
GET	`/api/pra/division-orders/stats`	Division order statistics
GET	`/api/pra/division-orders/property/:id`	Division orders by property
PUT	`/api/pra/division-orders/:id`	Update a division order

Royalty Runs

Method	Endpoint	Description
GET	`/api/pra/royalty-runs`	All royalty runs
GET	`/api/pra/royalty-runs/latest`	Latest royalty run
GET	`/api/pra/royalty-runs/:id`	Single royalty run
GET	`/api/pra/royalty-runs/:id/summary`	Royalty run summary
POST	`/api/pra/royalty-runs`	Create a royalty run

Payments

Method	Endpoint	Description
GET	`/api/pra/payments/:runId`	Payments for a royalty run
GET	`/api/pra/payments/:runId/stats`	Payment stats for a run

JVA — Joint Venture Accounting

Dashboard

Method	Endpoint	Description
GET	`/api/jva/dashboard`	Aggregated JVA dashboard (AFEs, elections, JIB stats, cash call stats, audit stats, receipts)

AFEs

Method	Endpoint	Description
GET	`/api/jva/afes`	All AFEs
GET	`/api/jva/afes/:id`	Single AFE by ID
POST	`/api/jva/afes`	Create a new AFE
PUT	`/api/jva/afes/:id`	Update an AFE
GET	`/api/jva/afes/:id/budget`	AFE budget breakdown with line items

AFE Line Items

Method	Endpoint	Description
GET	`/api/jva/afes/:id/lines`	Line items for an AFE
POST	`/api/jva/afes/:id/lines`	Add line item to AFE
PUT	`/api/jva/afe-lines/:id`	Update a line item

AFE Elections

Method	Endpoint	Description
GET	`/api/jva/elections`	All elections
GET	`/api/jva/elections/open`	Open elections awaiting response
GET	`/api/jva/afes/:id/elections`	Elections for a specific AFE
PUT	`/api/jva/elections/:id`	Respond to an election (participate/non_consent)

Cost Tracker

Method	Endpoint	Description
GET	`/api/jva/afes/:id/cost-tracker`	Budget vs actual with line items and elections

JIBs

Method	Endpoint	Description
GET	`/api/jva/jibs`	All JIBs
GET	`/api/jva/jibs/stats`	JIB statistics (total, outstanding, overdue)
GET	`/api/jva/jibs/:id`	Single JIB with line items
GET	`/api/jva/jibs/period/:period`	JIBs for a period
POST	`/api/jva/jibs`	Create a JIB
PUT	`/api/jva/jibs/:id`	Update a JIB

JIB Line Items

Method	Endpoint	Description
GET	`/api/jva/jibs/:id/lines`	Line items for a JIB
POST	`/api/jva/jibs/:id/lines`	Add line item to JIB

Cash Calls

Method	Endpoint	Description
GET	`/api/jva/cash-calls`	All cash calls
GET	`/api/jva/cash-calls/overdue`	Overdue cash calls
GET	`/api/jva/cash-calls/stats`	Cash call statistics
GET	`/api/jva/cash-calls/forecast`	90-day cash call forecast
POST	`/api/jva/cash-calls`	Create a cash call
PUT	`/api/jva/cash-calls/:id`	Update a cash call

Receipts

Method	Endpoint	Description
GET	`/api/jva/receipts`	All receipts
GET	`/api/jva/receipts/pending`	Pending/unapplied receipts
POST	`/api/jva/receipts`	Create a receipt
POST	`/api/jva/receipts/:id/apply`	Apply receipt to a cash call

Audit Findings

Method	Endpoint	Description
GET	`/api/jva/audit`	All audit findings

Method	Endpoint	Description
GET	`/api/jva/audit/stats`	Audit finding statistics
GET	`/api/jva/audit/:id`	Single finding by ID
POST	`/api/jva/audit`	Create an audit finding
PUT	`/api/jva/audit/:id`	Update an audit finding

---

6. 7-Step Onboarding Workflow

Step Summary

Step	Name	Key Data	Completion Criteria
1	Company Registration	Entity info, address, contacts	Legal name, EIN, 5 contacts verified
2	JOA & Working Interest	JOA terms, property assignments, WI%	WI decimal sum validates to 100%
3	Financial Setup	Banking, tax, payment preferences	ACH prenote confirmed, W-9 received
4	COPAS Compliance	Overhead elections, cost rules	Partner acknowledges COPAS terms
5	Document Collection	8 required documents	All documents received and verified
6	System Access	User accounts, MFA, role-based access	All accounts activated with MFA
7	Review & Submit	Readiness check, final approval	All steps complete, dual sign-off

Step 1: Company Registration

Purpose: Register the partner's legal entity, physical address, and designated contacts into the ConocoPhillips partner master database.

Required Data:

- Legal entity name and DBA/trade name
- Entity type (LLC, Corporation, LP, LLP, Trust, Individual)
- EIN/TIN (Employer Identification Number)
- D-U-N-S Number
- State and date of formation
- Registered street address (city, state, ZIP)
- 5 designated contacts:
    - Operations contact (primary)
    - Finance / CFO
    - Legal / General Counsel
    - Billing / Revenue Accountant
    - Elections / Land Manager

Validation Rules:

- Legal name is required
- Entity type must be one of the allowed values
- At least one primary contact must be designated

Step 2: JOA & Working Interest Configuration

Purpose: Configure the Joint Operating Agreement terms and assign working interest percentages to specific properties/wells.

Required Data:

- JOA reference number (e.g., JOA-2025-PB-042)
- Effective date
- Operator name (defaults to ConocoPhillips Company)

- Agreement type (AAPL Form 610)
- Election period (days)
- Non-consent penalty percentage
- Governing law jurisdiction
- Accounting procedure (COPAS version)
- Audit right window

Property Assignments (per property):

- Property/well name and basin
- Product type (OIL/GAS/NGL)
- Working Interest % (WI) — 6 decimal precision
- Net Revenue Interest % (NRI)
- Royalty burden %
- Decimal sum validation flag

Validation Rules:

- All WI percentages for a property must sum to 100.000000%
- NRI must be less than or equal to WI
- At least one property must be assigned

Step 3: Financial Setup

Purpose: Configure banking information for ACH/EFT payments, tax classification, and billing delivery preferences.

Required Data:

- Bank name, account type, routing number, masked account number
- Account holder name
- Payment method (ACH/EFT, Wire, Check)
- Tax classification (LLC, Corporation, etc.)
- 1099 reporting type (Box 2 Royalties, Box 7 NEC)
- Backup withholding status

- W-9 status and received date
- State severance tax ID
- ACH prenote status and initiation date
- Cash call delivery method
- JIB delivery method
- Payment terms (Net 30, Net 45)
- Revenue distribution method
- Cash call contact email
- AFE election contact email

Validation Rules:

- Routing number must be valid ABA format
- W-9 must be received before step can be marked complete
- ACH prenote must be initiated (3-5 day confirmation window)

Step 4: COPAS Compliance

Purpose: Configure COPAS (Council of Petroleum Accountants Societies) accounting elections that govern how operating costs are allocated between the operator and non-operator partners.

Required Data:

- COPAS version (e.g., 2005 MFI-51 Onshore)
- Overhead method (Fixed Rate per Well or Percentage of Cost)
- Escalation index (annual rate)
- Material pricing method (MPI)
- Audit period (months from JIB receipt)
- Dispute resolution mechanism

Overhead Rate Schedule:

Category	Typical Rate	Basis
Producing Well	$1,840/mo	Per well, per month
Non-Producing / SI	$920/mo	Per well, per month
Drilling Operations	10.0%	% of direct IDC costs
Major Construction	7.5%	% of direct costs

Cost Classification Rules:

Category	COPAS Code	Billable	Notes
Direct field labor	100	Yes	At actual salary + burden (cap 48%)
Supervisory labor	100	No	Recovered through overhead only
Materials at MPI	200	Yes	New: cost; Used: 75% of MPI
Contract services	400	Yes	Arm's-length rate for affiliates
G&A / corporate	N/A	No	Covered by overhead charge
Overhead	600	Yes	Per Schedule A or B election

Completion Criteria:

- Partner must formally acknowledge COPAS terms (acknowledged = 1)

Step 5: Document Collection

Purpose: Track receipt and verification of all required legal, financial, and regulatory documents.

8 Required Documents:

#	Document	Category	Purpose
1	W-9 (Request for TIN)	Tax	IRS tax ID verification
2	Certificate of Formation	Legal	Entity existence/good standing
3	Certificate of Insurance (COI)	Insurance	Liability coverage proof
4	Executed JOA Signature Pages	Legal	Binding agreement
5	ACH Authorization Form	Finance	Banking enrollment
6	Division Order (Executed)	Revenue	Revenue distribution authority
7	Operating Authority / RRC Filings	Regulatory	Texas Railroad Commission filings
8	COPAS Acknowledgment Letter	JVA	Overhead rate agreement

Document Lifecycle:

```
  pending → received → verified → complete
                                    ↑
  pending → overdue (past required_date) ──────┘ (when finally received +
verified)
```

Step 6: System Access & User Provisioning

Purpose: Create and activate user accounts in COP's partner portal systems with appropriate role-based access controls.

User Roles:

Role	Access Level	Systems
Partner Admin	Full Non-Op Access	All modules

Role	Access Level	Systems
Finance User	JIB + Cash Call + Revenue	Financial modules
Accounting User	JIB + Revenue Statements	Revenue reporting
Land User	AFE Elections + Division Order View	Land and elections
Legal / Audit	Audit Docs + JOA View	Document and agreement access

System Integrations:

- JV Partner Portal (SAP IS-Oil)
- EDI Gateway (JIB 810 + Cash Call 820)
- Mineral Connect
- AFE Election Portal
- Audit Document Access

Account Provisioning Flow:

```
queued → account created → MFA enrollment → active
```

Notification Preferences (per user):

- JIB notifications
- Cash call notifications
- AFE election alerts
- Revenue statement notifications

Step 7: Review & Submit for Approval

Purpose: Final readiness verification and dual sign-off before onboarding is marked complete.

Readiness Check — All must show "Complete":

1. Company Registration — entity and contacts verified
2. JOA & Working Interest — WI validated, JOA signed
3. Financial Setup — ACH confirmed, W-9 verified
4. COPAS Compliance — acknowledged by partner
5. Document Collection — all 8 documents complete
6. System Access — all accounts activated

Approval Requirements:

- Revenue Controller sign-off
- JV Manager sign-off
- Partner representative confirmation

---

7. Data Models

Model API Pattern

All 31 models follow a consistent API:

```
// Standard CRUD
Model.getAll()            // SELECT * (with optional JOINs)
Model.getById(id)         // SELECT ... WHERE id = ?
Model.create(data)        // INSERT ... RETURNING created row
Model.update(id, data)    // UPDATE ... dynamic field mapping
Model.delete(id)          // DELETE ... WHERE id = ?
```

Domain-Specific Methods

Model	Special Methods
Partner	`getFull(id)` — joins contacts, JOAs, financial, COPAS, users
JOA	`getWithInterests(partnerId)` — JOAs + properties + WI data
WorkingInterest	`validateDecimalSum(propertyId)` — verify WI sums to 100%
WorkingInterest	`getWISummary(partnerId)` — aggregate WI/NRI + COP split
Financial	`update()` with upsert pattern (create if not exists)

Model	Special Methods
COPAS	`acknowledge(partnerId)` — set acknowledged flag + date
COPAS	`getCostClassification()` — static COPAS cost rules
Document	`markReceived(id, date)` — transition pending → received
Document	`markVerified(id, date)` — transition received → complete
Document	`getStats(onboardingId)` — count by status
Onboarding	`updateStep(id, step, status)` — recalculates progress
Onboarding	`advanceStep(id, step)` — mark done, activate next step
Onboarding	`getStats()` — pipeline-level statistics
UserAccount	`activate(id)` — set status=active, mfa=enrolled

PRA Models

Model	Special Methods
PraPeriod	`getCurrent()` — latest open period; `updateChecklist()` — update close checklist JSON
ProductionVolume	`getByPeriod()`, `getExceptions()` — volumes with exceptions > 0
WellAllocation	`getByFacility()`, `getAlerts()` — stale tests or high variance
GasImbalance	`getAlerts()` — overlift/underlift/alert positions
PriceIndex	`lock(id)` — set status to locked with timestamp; `getHistory()` — 12-month trend
MarketingContract	`getExpiring()` — contracts within 90 days of expiration
Remittance	`getStats()` — count by match_status; `getExceptions()` — non-matched
SuspenseAccount	`getStats()` — totals, escheat risk count, avg days out
DivisionOrder	`getErrors()` — decimal sum != 1.0; `getStats()` — counts by status
RoyaltyRun	`getLatest()` — most recent run; `getSummary()` — run with payments

Model	Special Methods
OwnerPayment	`getByRun()`, `getStats()` — payment distribution breakdown

JVA Models

Model	Special Methods
AFE	`getBudgetComparison()` — authorized vs actual for all AFEs
AFELineItem	`getByAFE()` — line items grouped by COPAS code
AFEElection	`getOpen()` — pending elections; `respond()` — record election response
JIB	`getStats()` — total outstanding, overdue count, avg days; `getByPeriod()`
JIBLineItem	`getByJIB()` — line items for a JIB
CashCall	`getOverdue()`, `getStats()`, `getForecast()` — 90-day cash requirements
Receipt	`getPending()` — unapplied; `apply()` — match receipt to cash call
AuditFinding	`getStats()` — count by status, total amounts

Progress Calculation

```
overall_progress = round(((done_steps * 100) + (active_steps * 45)) / 7)
```

- A "done" step contributes 100 points
- An "active" step contributes 45 points (roughly halfway)
- Divided by 7 total steps and rounded

---

8. Security & Compliance

Data Protection

- Account numbers are stored masked (e.g., ****4872)

- No plaintext passwords stored (portal provisioning is external)
- Foreign keys with ON DELETE CASCADE prevent orphaned records

Oil & Gas Regulatory Context

- **AAPL Form 610 (2015)** — Standard JOA form used industry-wide
- **COPAS 2005 MFI-51** — Model Form Interpretation for onshore accounting
- **Texas Railroad Commission (RRC)** — Regulatory filings for Texas operations
- **IRS W-9** — Required for all payment recipients
- **Division Orders** — Legal instrument authorizing revenue distributions

Audit Trail

- All onboarding actions logged to `activity_log` with timestamps
- Document receipt and verification dates tracked separately
- COPAS acknowledgment date captured
- Step advancement logged with step name and timestamp

---

9. Setup & Deployment

Prerequisites

- Node.js 18+ installed

Installation

```
cd COPProject
npm install
node server.js
```

The server starts on **http://localhost:3000**. On first run, the SQLite database is automatically created and seeded with partner, PRA, and JVA data.

Seed Data

The system ships with comprehensive seed data across all three domains:

- 7 partners (6 onboarding + ConocoPhillips Company as operator)
- 10 properties across 4 JOAs
- 12 months of price index history (WTI, Henry Hub, Mont Belvieu)
- 4 AFEs with line items and election tracking
- 4 JIBs with line items
- 4 cash calls and 3 receipts
- 5 audit findings
- Suspense accounts, division orders, royalty run with owner payments
- Production volumes and well allocations
- Gas imbalance records
- Marketing contracts with remittance matching

Onboarding Partners:

Partner	Ref	Type	Progress	Properties
Sierra Digital LLC	ONB-2025-0018	Non-Operator	42%	Permian Spr #42-44
Apex Resources Inc.	ONB-2025-0017	Non-Operator	85%	EF Karnes #B3
Basin Energy Corp	ONB-2025-0016	Non-Operator	60%	Montney #28
Ridgeline Petroleum LLC	ONB-2025-0015	Operator Transfer	30%	Poseidon Deep
Falcon Exploration Inc.	ONB-2025-0014	Non-Operator	15%	Permian Mid #22
Catalyst JV Partners LP	ONB-2025-0013	Farm-In	95%	EF DeWitt #11

Database Reset

To reset to initial state, delete the database file and restart:

```
rm database/onboarding.db
node server.js
```

---

10. PRA — Product Revenue Accounting Module

Purpose

Monthly cycle that tracks production volumes from wellhead through purchaser remittance to royalty distribution. Ensures accurate revenue recognition under ASC 606/932.

Monthly Close Process (8-step checklist)

1. **Volume Close** — Lock SCADA/gauge volumes
2. **Facility Balance** — Validate well allocation factors
3. **Price Lock** — Lock commodity price indices (WTI, Henry Hub, Mont Belvieu)
4. **Remittance Match** — Reconcile purchaser statements
5. **Gas Balance** — Review imbalance positions
6. **Royalty Calc** — Calculate owner distributions
7. **Owner Statements** — Generate and distribute statements
8. **GL Posting** — Post revenue entries

Key Formulas

- **Net Revenue** = Gross Volume x NRI x Net Price
- **NRI** = WI% x (1 - Royalty Burden)
- **Gas Imbalance** = Entitlement MCF - Taken MCF
- **Remittance Variance** = (Our Vol - Purch Vol) / Our Vol x 100

Alert Thresholds

- **Volume exceptions:** any property with exceptions > 0
- **Allocation variance:** > 2% triggers review
- **Stale well tests:** > 30 days
- **Gas imbalance:** overlift/underlift positions

- **Suspense aging:** > 180 days triggers escheat risk
- **Contract expiration:** within 90 days

---

11. JVA — Joint Venture Accounting Module

Purpose

Manage operator/non-operator cost sharing per COPAS 2005 MFI-51. Tracks AFE authorization, JIB billing, cash call collections, and audit compliance.

AFE Lifecycle

1. Operator creates AFE with budget by COPAS code (100-700)
2. Elections sent to non-operators (30-day response window)
3. Partners respond: Participate or Non-Consent (300% penalty)
4. Cost tracking: Authorized vs Actual spend
5. Supplemental AFE if overrun > threshold

JIB Process

1. Accumulate direct costs by COPAS code
2. Apply overhead rates per COPAS schedule
3. Calculate non-operator share (cost x WI%)
4. Issue JIB with 30-day payment terms
5. Track payment status (issued/partial/paid/overdue)

Cash Call Cycle

1. Issue cash call for upcoming AFE activity
2. Track due date and received amounts
3. Calculate outstanding balance and days past due
4. Accrue interest on overdue amounts
5. Receipt application: match payments to cash calls

COPAS Code Categories

Code	Category	Description
100	Labor	Direct field labor at actual + burden (cap 48%)
200	Materials	New: cost; Used: 75% of MPI
300	Transportation	Equipment and material hauling
400	Contract Services	Third-party services at arm's-length rates
600	Overhead	Per Schedule A (fixed) or B (percentage)
700	Other	Taxes, permits, environmental

Audit Process

- 24-month audit window from JIB receipt
- Categories: overhead charge, material pricing, AFE scope, labor rate, transportation
- Finding statuses: open, disputed, resolved, credit_due